

APIs used in Common Gateway

SDK Platform Abstraction API (sdk_*) — Developer Reference

Standard return type – SdkResult (0 = success, negative = failure):

Code	Name	Meaning
0	SDK_RESULT_SUCCESS	operation succeeded
-1	SDK_RESULT_ERROR	generic failure
-2	SDK_RESULT_TIMEOUT	operation timed out
-3	SDK_RESULT_INVALID_PARAM	bad / NULL argument
-4	SDK_RESULT_NOT_INITIALIZED	subsystem not initialised
-8	SDK_RESULT_BUSY	retry later (in progress)
-9	SDK_RESULT_NOT_SUPPORTED	vendor does not implement it

NETWORK

sdk_network_set_cfun(UINT32 cfun)

Description: Set the modem radio functionality level (AT+CFUN).

Parameters:

UINT32 cfun - functionality level: 0=minimum, 1=full, 4=airplane/RF-off.

Returns: SdkResult - 0 success; negative on failure.

sdk_network_get_cfun(UINT32* cfun)

Description: Read the current CFUN level.

Parameters:

UINT32* cfun - [out] receives the current level.

Returns: SdkResult - 0 success; negative on failure.

sdk_network_get_creg(SdkNetRegStatus* status)

Description: Get circuit-switched (voice) network registration status (AT+CREG).

Parameters:

SdkNetRegStatus* status - [out] registration state (home/roaming/searching/denied).

Returns: SdkResult - 0 success; negative on failure.

sdk_network_get_cgreg(SdkNetRegStatus* status)

Description: Get GPRS/packet-switched registration status (AT+CGREG).

Parameters:

SdkNetRegStatus* status - [out] PS registration state.

Returns: SdkResult - 0 success; negative on failure.

sdk_network_get_cgatt(SdkNetAttStatus* status)

Description: Get packet-domain attach status (AT+CGATT).

Parameters:

SdkNetAttStatus* status - [out] attached / detached.

Returns: SdkResult - 0 success; negative on failure.

sdk_network_set_pdp_context(UINT32 cid, const char* apn_type, const char* apn)

Description: Define a PDP context (APN) used for the data session (AT+CGDCONT).

Parameters:

UINT32 cid - context id (e.g. 1).

const char* apn_type - PDP type string, e.g. "IP" / "IPV4V6".

const char* apn - access point name (e.g. "wheelsseye.com").

Returns: SdkResult - 0 success; negative on failure.

sdk_network_activate_pdp(UINT32 activate, UINT32 cid)

Description: Activate or deactivate a PDP context (AT+CGACT).

Parameters:

UINT32 activate - 1=activate, 0=deactivate.

UINT32 cid - context id to (de)activate.

Returns: SdkResult - 0 success; negative on failure.

sdk_network_get_ip_address(UINT32 cid, SdkIpAddress* ip_addr)

Description: Get the IP address assigned to a PDP context (AT+CGPADDR).

Parameters:

UINT32 cid - context id.

SdkIpAddress* ip_addr - [out] assigned IP address.

Returns: SdkResult - 0 success; negative on failure.

sdk_network_get_network_status(void)

Description: Query overall network status.

Returns: SdkResult - 0 = up/registered; negative otherwise.

sdk_network_get_sim_pin_status(UINT8* pin_status)

Description: Get SIM PIN/lock status via the network stack (AT+CPIN).

Parameters:

UINT8* pin_status - [out] pin-status code (0 = ready).

Returns: SdkResult - 0 success; negative on failure.

sdk_network_get_gps_radio_info(SdkNetworkGpsRadioInfo* info)

Description: Get radio info for the GPS packet (signal + serving cell).

Parameters:

SdkNetworkGpsRadioInfo* info - [out] CSQ, RSRP/RSRQ, MCC, MNC, LAC, cell id, valid flag.

Returns: SdkResult - 0 success; negative on failure.

sdk_network_get_time(SdkNetworkTime* time)

Description: Get network-provided time (NITZ).

Parameters:

SdkNetworkTime* time - [out] network date/time.

Returns: SdkResult - 0 success; negative on failure.

sdk_network_set_rtc(const SdkNetworkTime* time)

Description: Set the modem real-time clock.

Parameters:

const SdkNetworkTime* time - time value to write.

Returns: SdkResult - 0 success; negative on failure.

sdk_network_rtc_get_local_time(SdkNetworkTime* time)

Description: Read the modem RTC in local time.

Parameters:

SdkNetworkTime* time - [out] local time.

Returns: SdkResult - 0 success; negative on failure.

sdk_network_rtc_get_utc_time(SdkNetworkTime* time)

Description: Read the modem RTC in UTC.

Parameters:

SdkNetworkTime* time - [out] UTC time.

Returns: SdkResult - 0 success; negative on failure.

sdk_network_get_ctzu(UINT32* ctzu)

Description: Get automatic time-zone-update mode (AT+CTZU).

Parameters:

UINT32* ctzu - [out] 0=off, 1=on.

Returns: SdkResult - 0 success; negative on failure.

sdk_network_set_ctzu(UINT32 enable)

Description: Enable/disable automatic time-zone update (AT+CTZU).

Parameters:

UINT32 enable - 1=enable, 0=disable.

Returns: SdkResult - 0 success; negative on failure.

SIM

sdk_sim_get_status(SdkSimStatus* status)

Description: Get SIM presence/ready status.

Parameters:

SdkSimStatus* status - [out] SIM state (present/absent/ready/error).

Returns: SdkResult - 0 success; negative on failure.

sdk_sim_get_pin_status(UINT8* pin_status)

Description: Get SIM PIN lock state.

Parameters:

UINT8* pin_status - [out] pin-status code (0 = ready, no PIN).

Returns: SdkResult - 0 success; negative on failure.

sdk_sim_get_iccid(char* iccid, UINT32 iccid_size)

Description: Read the SIM ICCID (unique SIM serial number).

Parameters:

char* iccid - [out] buffer for the ICCID string.

UINT32 iccid_size - size of the buffer in bytes.

Returns: SdkResult - 0 success; negative on failure.

sdk_sim_hotswap_msg(UINT32 command, UINT32 value, void* msgq)

Description: Configure SIM hot-swap (insert/remove) detection; events delivered async.

Parameters:

UINT32 command - hot-swap command/config selector.

UINT32 value - command value.

void* msgq - message queue that will receive hot-swap events.

Returns: SdkResult - 0 success; negative on failure.

sdk_sim_hotswap_poll(void* msgq)

Description: Poll the hot-swap message queue for SIM insert/remove events.

Parameters:

void* msgq - the message queue to poll.

Returns: SdkResult - 0 success; negative on failure.

SMS

sdk_sms_set_format(UINT8 format)

Description: Set SMS message format (AT+CMGF).

Parameters:

UINT8 format - 0=PDU mode, 1=text mode.

Returns: SdkResult - 0 success; negative on failure.

sdk_sms_set_charset(UINT8 charset)

Description: Set SMS character set (AT+CSCS).

Parameters:

UINT8 charset - character-set selector (e.g. GSM/UCS2/IRA).

Returns: SdkResult - 0 success; negative on failure.

sdk_sms_set_new_msg_ind(UINT8 mode, UINT8 mt, UINT8 bm, UINT8 ds, UINT8 bfr)

Description: Configure new-message URC indication (AT+CNMI).

Parameters:

UINT8 mode, mt, bm, ds, bfr - the five CNMI parameters controlling how incoming-message notifications are delivered/buffered.

Returns: SdkResult - 0 success; negative on failure.

sdk_sms_read(UINT8 storage, UINT32 index, void* msgq)

Description: Read a stored SMS by index (result delivered via msgq).

Parameters:

UINT8 storage - storage area selector.

UINT32 index - message index to read.

void* msgq - queue to receive the read result.

Returns: SdkResult - 0 success; negative on failure.

sdk_sms_delete(UINT32 index, void* msgq)

Description: Delete a stored SMS by index.

Parameters:

UINT32 index - message index to delete.

void* msgq - queue to receive the result.

Returns: SdkResult - 0 success; negative on failure.

sdk_sms_delete_all(void* msgq)

Description: Delete all stored SMS.

Parameters:

void* msgq - queue to receive the result.

Returns: SdkResult - 0 success; negative on failure.

sdk_sms_msgq_poll(void* msgq, UINT32* msg_count)

Description: Poll the SMS message queue for pending messages.

Parameters:

void* msgq - the SMS message queue.

UINT32* msg_count - [out] number of messages pending.

Returns: SdkResult - 0 success; negative on failure.

GPS / GNSS

sdk_gps_get_power_status(UINT8* power_on)

Description: Get GNSS receiver power state.

Parameters:

UINT8* power_on - [out] 1=on, 0=off.

Returns: SdkResult - 0 success; negative on failure.

sdk_gps_set_power_status(UINT8 power_on)

Description: Power the GNSS receiver on or off.

Parameters:

UINT8 power_on - 1=power on, 0=power off.

Returns: SdkResult - 0 success; negative on failure.

sdk_gps_set_mode(UINT32 mode)

Description: Set the GNSS constellation mode (e.g. GPS, GPS+GLONASS).

Parameters:

UINT32 mode - constellation-mode selector.

Returns: SdkResult - 0 success; negative on failure.

sdk_gps_set_nmea_rate(UINT32 rate)

Description: Set the NMEA sentence output rate.

Parameters:

UINT32 rate - output rate (1 = 1 Hz).

Returns: SdkResult - 0 success; negative on failure.

sdk_gps_start_mode(UINT32 mode)

Description: Set the GNSS start mode.

Parameters:

UINT32 mode - 0=HOT, 1=WARM, 2=COLD start.

Returns: SdkResult - 0 success; negative on failure.

sdk_gps_enable_nmea_output(UINT32 data_get_mode)

Description: Start the NMEA output stream so location data begins flowing.

Parameters:

UINT32 data_get_mode - 0=deliver via serial port, 1=deliver via URC.

Returns: SdkResult - 0 success; negative on failure.

sdk_gps_set_gnss_info_period(UINT32 period_sec)

Description: Set the periodic GNSS info report interval.

Parameters:

UINT32 period_sec - report period in seconds (0-255).

Returns: SdkResult - 0 success; negative on failure.

sdk_gps_open_agps_service(void)

Description: Open the A-GPS assistance service to speed up first fix.

Returns: SdkResult - 0 success; negative on failure.

sdk_gps_set_ap_flash_hot_start(BOOL enable)

Description: Enable/disable saving ephemeris to AP flash for faster hot starts.

Parameters:

BOOL enable - TRUE=enable, FALSE=disable.

Returns: SdkResult - 0 success; negative on failure.

sdk_gps_get_navdata(SdkGpsNavData* nav_data)

Description: Get parsed navigation data from the receiver.

Parameters:

SdkGpsNavData* nav_data - [out] fix flag, lat, lon, altitude, speed, course, sats, etc.

Returns: SdkResult - 0 success; negative on failure.

TCP (lwIP / BSD sockets)

Note: these follow BSD socket conventions, NOT SdkResult.

sdk_tcp_socket_create_with_callback(int af, int type, int protocol, SdkTcpSocketCallback callback)

Description: Create a TCP socket and register an async event callback.

Parameters:

int af - address family (e.g. AF_INET).

int type - socket type (e.g. SOCK_STREAM).

int protocol - protocol (0 = default TCP).

SdkTcpSocketCallback callback - fired on socket events (connect/recv/close).

Returns: int - socket fd (>= 0) on success; < 0 on error.

sdk_tcp_connect(int fd, const void* addr, unsigned int addrlen)

Description: Connect a socket to a remote address.

Parameters:

int fd - socket fd.

const void* addr - remote sockaddr pointer.

unsigned int addrlen - size of the address struct.

Returns: int - 0 on success; < 0 on error.

sdk_tcp_send(int fd, const void* buf, unsigned int len, int flags)

Description: Send bytes on a connected socket.

Parameters:

int fd - socket fd.

const void* buf - data to send.

unsigned int len - number of bytes.

int flags - send flags (usually 0).

Returns: int - bytes sent (>= 0); < 0 on error.

sdk_tcp_recv(int fd, void* buf, unsigned int len, int flags)

Description: Receive bytes from a socket.

Parameters:

int fd - socket fd.

void* buf - [out] receive buffer.

unsigned int len - buffer size.

int flags - recv flags (usually 0).

Returns: int - bytes received (>0); 0 = peer closed; < 0 on error.

sdk_tcp_close(int fd)

Description: Close a socket and release its resources.

Parameters:

int fd - socket fd.

Returns: int - 0 on success; < 0 on error.

sdk_tcp_get_errno(void)

Description: Get the last global socket error number.

Returns: int - errno value.

sdk_tcp_get_sock_errno(int fd)

Description: Get the pending error (SO_ERROR) for a specific socket.

Parameters:

int fd - socket fd.

Returns: int - errno value for that socket.

HTTPS

sdk_https_init(UINT32 tx_mode, void* msgq)

Description: Initialise the HTTPS client (async results via a message queue).

Parameters:

UINT32 tx_mode - transfer/response delivery mode.
void* msgq - queue to receive HTTP events/results.

Returns: SdkResult - 0 success; negative on failure.

sdk_https_configure_ssl(UINT32 ssl_index, const char* ca_cert, const char* client_cert, const char* client_key)

Description: Configure the SSL/TLS certificates for a session.

Parameters:

UINT32 ssl_index - SSL context index.
const char* ca_cert - CA certificate (PEM) or NULL.
const char* client_cert - client cert or NULL.
const char* client_key - client key or NULL.

Returns: SdkResult - 0 success; negative on failure.

sdk_https_set_params(UINT32 ssl_index, const char* url, UINT32 timeout)

Description: Set the request URL and timeout.

Parameters:

UINT32 ssl_index - SSL context index.
const char* url - request URL.
UINT32 timeout - timeout in seconds.

Returns: SdkResult - 0 success; negative on failure.

sdk_https_set_data(UINT32 ssl_index, const char* data, UINT32 data_len)

Description: Set the request body/payload data.

Parameters:

UINT32 ssl_index - SSL context index.
const char* data - payload bytes.
UINT32 data_len - payload length.

Returns: SdkResult - 0 success; negative on failure.

sdk_https_action(UINT32 ssl_index, UINT32 action)

Description: Perform an HTTP action (GET/POST/etc.).

Parameters:

UINT32 ssl_index - SSL context index.
UINT32 action - action selector (0=GET, 1=POST, ...).

Returns: SdkResult - 0 success; negative on failure.

sdk_https_read(UINT32 ssl_index, void* buffer, UINT32 size, UINT32* bytes_read)

Description: Read response body bytes.

Parameters:

UINT32 ssl_index - SSL context index.
void* buffer - [out] response buffer.
UINT32 size - buffer size.
UINT32* bytes_read - [out] bytes actually read.

Returns: SdkResult - 0 success; negative on failure.

sdk_https_get_response(void)

Description: Get a pointer to the last response buffer.

Returns: const char* - pointer to response bytes (NULL if none).

sdk_https_get_response_len(void)

Description: Get the length of the last response.

Returns: UINT32 - response length in bytes (0 if none).

sdk_https_terminate(UINT32 ssl_index)

Description: Terminate the HTTPS session and free resources.

Parameters:

UINT32 ssl_index - SSL context index.

Returns: SdkResult - 0 success; negative on failure.

sdk_https_download_configure_ssl(void)

Description: Configure SSL specifically for the file-download path.

Returns: int - 0 on success; < 0 on error.

sdk_https_download_get_file_size(UINT32* file_size)

Description: Query the size of the remote file to download.

Parameters:

UINT32* file_size - [out] file size in bytes.

Returns: int - 0 on success; < 0 on error.

sdk_https_download_read_chunk(UINT32 offset, UINT32 size, char* buffer, UINT32* bytes_read)

Description: Read one chunk of the file being downloaded.

Parameters:

UINT32 offset - byte offset to read from.

UINT32 size - number of bytes to read.

char* buffer - [out] chunk buffer.

UINT32* bytes_read - [out] bytes actually read.

Returns: int - 0 on success; < 0 on error.

OTA

sdk_ota_download_init(UINT32 tx_mode, void* msgq)

Description: Initialise the OTA firmware download (async via msgq).

Parameters:

UINT32 tx_mode - delivery mode.

void* msgq - queue to receive OTA events.

Returns: SdkResult - 0 success; negative on failure.

sdk_ota_download_configure_ssl(UINT32 ssl_index, const char* ca_cert)

Description: Set the CA certificate for the OTA download server.

Parameters:

UINT32 ssl_index - SSL context index.

const char* ca_cert - CA certificate (PEM).

Returns: SdkResult - 0 success; negative on failure.

sdk_ota_download_set_params(UINT32 ssl_index, const char* url, UINT32 timeout)

Description: Set the OTA image URL and timeout.

Parameters:

UINT32 ssl_index - SSL context index.

const char* url - firmware image URL.

UINT32 timeout - timeout in seconds.

Returns: SdkResult - 0 success; negative on failure.

sdk_ota_download_action(UINT32 ssl_index)

Description: Start the OTA download.

Parameters:

UINT32 ssl_index - SSL context index.

Returns: SdkResult - 0 success; negative on failure.

sdk_ota_download_get_file_size(UINT32 ssl_index, UINT32* file_size)

Description: Get the OTA image size.

Parameters:

UINT32 ssl_index - SSL context index.

UINT32* file_size - [out] image size in bytes.

Returns: SdkResult - 0 success; negative on failure.

sdk_ota_download_read_chunk(UINT32 ssl_index, void* buffer, UINT32 size, UINT32* bytes_read)

Description: Read one chunk of the OTA image.

Parameters:

UINT32 ssl_index - SSL context index.

void* buffer - [out] chunk buffer.

UINT32 size - bytes to read.

UINT32* bytes_read - [out] bytes actually read.

Returns: SdkResult - 0 success; negative on failure.

sdk_ota_download_terminate(UINT32 ssl_index)

Description: End the OTA download session.

Parameters:

UINT32 ssl_index - SSL context index.
Returns: SdkResult - 0 success; negative on failure.

sdk_app_package_open(const char* mode)

Description: Open the firmware application package for writing.

Parameters:

const char* mode - open mode string (e.g. "w").

Returns: SdkResult - 0 success; negative on failure.

sdk_app_package_read(void* data, UINT32 size, UINT32* bytes_read)

Description: Read from the app package.

Parameters:

void* data - [out] buffer.

UINT32 size - bytes to read.

UINT32* bytes_read - [out] bytes actually read.

Returns: SdkResult - 0 success; negative on failure.

sdk_app_package_write(const void* data, UINT32 size)

Description: Write firmware bytes into the app package.

Parameters:

const void* data - firmware bytes.

UINT32 size - number of bytes.

Returns: SdkResult - 0 success; negative on failure.

sdk_app_package_close(void)

Description: Close the app package (finalise write).

Returns: SdkResult - 0 success; negative on failure.

sdk_ota_fbf_disable(void)

Description: Disable the fallback/backup-firmware feature.

Returns: SdkResult - 0 success; negative on failure.

UART

sdk_uart_set_config(UINT32 port, const SdkUartConfig* config)

Description: Configure a UART port (baud, data bits, stop, parity, flow control).

Parameters:

UINT32 port - UART port id.

const SdkUartConfig* config - configuration structure.

Returns: SdkResult - 0 success; negative on failure.

sdk_uart_read(UINT32 port, void* buffer, UINT32 size, UINT32* bytes_read)

Description: Read bytes from a UART port.

Parameters:

UINT32 port - UART port id.

void* buffer - [out] receive buffer.

UINT32 size - buffer size.

UINT32* bytes_read - [out] bytes actually read.

Returns: SdkResult - 0 success; negative on failure.

sdk_uart_write(UINT32 port, const void* buffer, UINT32 size, UINT32* bytes_written)

Description: Write bytes to a UART port.

Parameters:

UINT32 port - UART port id.

const void* buffer - data to send.

UINT32 size - number of bytes.

UINT32* bytes_written - [out] bytes actually written.

Returns: SdkResult - 0 success; negative on failure.

sdk_uart_control(UINT32 port, UINT32 command)

Description: Perform a UART control operation (register callback / flush / etc.).

Parameters:

UINT32 port - UART port id.

UINT32 command - control command selector.

Returns: SdkResult - 0 success; negative on failure.

FILE SYSTEM

sdk_file_open(const char* path, const char* mode)

Description: Open a file.

Parameters:

const char* path - file path.
const char* mode - open mode ("r", "w", "a", "rb", ...).

Returns: void* - file handle on success; NULL on failure.

sdk_file_close(void* file)

Description: Close an open file handle.

Parameters:

void* file - file handle.

Returns: SdkResult - 0 success; negative on failure.

sdk_file_read(void* file, void* buffer, UINT32 size, UINT32* bytes_read)

Description: Read bytes from a file.

Parameters:

void* file - file handle.
void* buffer - [out] read buffer.
UINT32 size - bytes to read.
UINT32* bytes_read - [out] bytes actually read.

Returns: SdkResult - 0 success; negative on failure.

sdk_file_write(void* file, const void* buffer, UINT32 size, UINT32* bytes_written)

Description: Write bytes to a file.

Parameters:

void* file - file handle.
const void* buffer - data to write.
UINT32 size - number of bytes.
UINT32* bytes_written - [out] bytes actually written.

Returns: SdkResult - 0 success; negative on failure.

sdk_file_seek(void* file, INT32 offset, UINT32 whence)

Description: Move the file read/write position.

Parameters:

void* file - file handle.
INT32 offset - offset value.
UINT32 whence - origin (SEEK_SET/CUR/END equivalent).

Returns: SdkResult - 0 success; negative on failure.

sdk_file_sync(void* file)

Description: Flush buffered file data to persistent storage.

Parameters:

void* file - file handle.

Returns: SdkResult - 0 success; negative on failure.

sdk_file_get_size(void* file, UINT32* size)

Description: Get the size of an open file.

Parameters:

void* file - file handle.
UINT32* size - [out] size in bytes.

Returns: SdkResult - 0 success; negative on failure.

sdk_file_exists(const char* path)

Description: Check whether a path exists.

Parameters:

const char* path - path to check.

Returns: SdkResult - 0 = exists; negative = not found/error.

sdk_file_delete(const char* path)

Description: Delete a file.

Parameters:

const char* path - file path.

Returns: SdkResult - 0 success; negative on failure.

sdk_file_rename(const char* old_path, const char* new_path)

Description: Rename or move a file.

Parameters:

const char* old_path - existing path.

const char* new_path - new path.

Returns: SdkResult - 0 success; negative on failure.

sdk_file_mkdir(const char* path)

Description: Create a directory.

Parameters:

const char* path - directory path.

Returns: SdkResult - 0 success; negative on failure.

STORAGE (NVM)

sdk_storage_read_nvm(UINT32 partition, UINT32 offset, void* buffer, UINT32 size)

Description: Read raw bytes from an NVM flash partition.

Parameters:

UINT32 partition - partition id.

UINT32 offset - byte offset within the partition.

void* buffer - [out] read buffer.

UINT32 size - bytes to read.

Returns: SdkResult - 0 success; negative on failure.

sdk_storage_write_nvm(UINT32 partition, UINT32 offset, const void* buffer, UINT32 size)

Description: Write raw bytes to an NVM flash partition.

Parameters:

UINT32 partition - partition id.

UINT32 offset - byte offset within the partition.

const void* buffer - data to write.

UINT32 size - number of bytes.

Returns: SdkResult - 0 success; negative on failure.

OS / RTOS

sdk_task_create(void (*task_fn)(void*), void* arg, const char* name, void* stack_ptr, UINT32 stack_size, UINT32 priority)

Description: Create and start an RTOS task.

Parameters:

void (*task_fn)(void*) - task entry function.

void* arg - argument passed to the task.

const char* name - task name (for debug).

void* stack_ptr - caller-provided stack buffer.

UINT32 stack_size - stack size in bytes.

UINT32 priority - task priority.

Returns: void* - task handle on success; NULL on failure.

sdk_task_delete(void* task_handle)

Description: Delete a task.

Parameters:

void* task_handle - handle from sdk_task_create.

Returns: SdkResult - 0 success; negative on failure.

sdk_task_sleep(UINT32 ms)

Description: Sleep/yield the current task.

Parameters:

UINT32 ms - sleep duration in milliseconds.

Returns: void.

sdk_task_get_stack_info(void* task_handle, UINT32* stack_size, UINT32* stack_used, UINT32* stack_peak)

Description: Get stack usage statistics for a task.

Parameters:

void* task_handle - task handle.
UINT32* stack_size - [out] total stack size.
UINT32* stack_used - [out] currently used.
UINT32* stack_peak - [out] peak usage.

Returns: SdkResult - 0 success; negative on failure.

sdk_get_ticks(void)

Description: Get the millisecond system tick counter.

Returns: UINT32 - milliseconds since boot (wraps).

sdk_mutex_create(void mutex, UINT32 wait_mode)**

Description: Create a mutex.

Parameters:

void** mutex - [out] receives the mutex handle.
UINT32 wait_mode - queuing/wait mode.

Returns: SdkResult - 0 success; negative on failure.

sdk_mutex_lock(void* mutex, UINT32 timeout)

Description: Acquire a mutex.

Parameters:

void* mutex - mutex handle.
UINT32 timeout - wait timeout (ms).

Returns: SdkResult - 0 success; -2 timeout; negative on other failure.

sdk_mutex_unlock(void* mutex)

Description: Release a mutex.

Parameters:

void* mutex - mutex handle.

Returns: SdkResult - 0 success; negative on failure.

sdk_mutex_delete(void* mutex)

Description: Delete a mutex.

Parameters:

void* mutex - mutex handle.

Returns: SdkResult - 0 success; negative on failure.

sdk_msgq_create(const char* name, UINT32 msg_size, UINT32 capacity, UINT32 flags)

Description: Create a message queue.

Parameters:

const char* name - queue name.
UINT32 msg_size - size of one message.
UINT32 capacity - number of messages.
UINT32 flags - creation flags.

Returns: void* - queue handle on success; NULL on failure.

sdk_msgq_send(void* msgq, const void* msg, UINT32 timeout_ms)

Description: Send a message to a queue.

Parameters:

void* msgq - queue handle.
const void* msg - message to send.
UINT32 timeout_ms - wait timeout if queue full.

Returns: SdkResult - 0 success; -2 timeout; negative on failure.

sdk_msgq_rcv(void* msgq, void* msg, UINT32 timeout_ms)

Description: Receive a message from a queue.

Parameters:

void* msgq - queue handle.
void* msg - [out] received message.
UINT32 timeout_ms - wait timeout if empty.

Returns: SdkResult - 0 success; -2 timeout; negative on failure.

sdk_msgq_delete(void* msgq)

Description: Delete a message queue.

Parameters:

void* msgq - queue handle.

Returns: SdkResult - 0 success; negative on failure.

sdk_memory_alloc(UINT32 size)

Description: Allocate heap memory.

Parameters:

UINT32 size - number of bytes.

Returns: void* - pointer on success; NULL on failure.

sdk_memory_free(void* ptr)

Description: Free heap memory.

Parameters:

void* ptr - pointer from sdk_memory_alloc.

Returns: void.

sdk_system_get_stats(UINT32* ram_total_kb, UINT32* ram_free_kb, INT64* flash_total_kb, INT64* flash_free_kb, UINT8* cpu_usage_percent)

Description: Get RAM/flash/CPU usage statistics.

Parameters:

UINT32* ram_total_kb, UINT32* ram_free_kb - [out] RAM totals/free (KB).

INT64* flash_total_kb, INT64* flash_free_kb - [out] flash totals/free (KB).

UINT8* cpu_usage_percent - [out] CPU load %.

Returns: SdkResult - 0 success; negative on failure.

DEVICE

sdk_device_get_imei(char* imei, UINT32 imei_size)

Description: Read the device IMEI (15-digit modem identity).

Parameters:

char* imei - [out] buffer for the IMEI string.

UINT32 imei_size - buffer size (>=16 recommended).

Returns: SdkResult - 0 success; negative on failure.

GPIO

sdk_gpio_set_direction(UINT32 pin, UINT32 direction)

Description: Set a GPIO pin as input or output.

Parameters:

UINT32 pin - pin number.

UINT32 direction - 0=input, 1=output (per platform).

Returns: SdkResult - 0 success; negative on failure.

sdk_gpio_get_direction(UINT32 pin, UINT32* direction)

Description: Read a GPIO pin's configured direction.

Parameters:

UINT32 pin - pin number.

UINT32* direction - [out] current direction.

Returns: SdkResult - 0 success; negative on failure.

sdk_gpio_set_level(UINT32 pin, UINT32 level)

Description: Set the output level of a GPIO pin.

Parameters:

UINT32 pin - pin number.

UINT32 level - 0=low, 1=high.

Returns: SdkResult - 0 success; negative on failure.

sdk_gpio_get_level(UINT32 pin, UINT32* level)

Description: Read the current level of a GPIO pin.

Parameters:

UINT32 pin - pin number.

UINT32* level - [out] 0=low, 1=high.

Returns: SdkResult - 0 success; negative on failure.

ADC

sdk_adc_read_voltage(INT32 channel, UINT16* voltage_mv)

Description: Read the voltage on an ADC channel.

Parameters:

UINT32 channel - ADC channel number.
UINT16* voltage_mv - [out] voltage in millivolts.

Returns: SdkResult - 0 success; negative on failure.

sdk_adc_read_vbat_voltage(UINT16* vbat_mv)

Description: Read the main battery (VBAT) voltage.

Parameters:

UINT16* vbat_mv - [out] VBAT voltage in millivolts.

Returns: SdkResult - 0 success; negative on failure.

I2C

sdk_i2c_hw_init(UINT32 channel, SdkI2cSpeed speed)

Description: Initialise a hardware I2C channel.

Parameters:

UINT32 channel - I2C channel id.
SdkI2cSpeed speed - bus speed (standard 100k / fast 400k).

Returns: SdkResult - 0 success; negative on failure.

sdk_i2c_hw_deinit(UINT32 channel)

Description: Deinitialise an I2C channel.

Parameters:

UINT32 channel - I2C channel id.

Returns: SdkResult - 0 success; negative on failure.

URC

sdk_urc_register(void* msgq, UINT32 mask)

Description: Register a message queue to receive unsolicited result codes (URCs).

Parameters:

void* msgq - queue to receive URCs.
UINT32 mask - bitmask selecting which URC types to receive.

Returns: SdkResult - 0 success; negative on failure.

sdk_urc_unregister(void* msgq)

Description: Unregister a previously registered URC queue.

Parameters:

void* msgq - the queue to unregister.

Returns: SdkResult - 0 success; negative on failure.

SYSTEM

sdk_system_reset(void)

Description: Perform an immediate system reset.

Returns: void (does not return).

sdk_system_reboot(void)

Description: Reboot the device (graceful).

Returns: void (does not return).

sdk_system_power_off(void)

Description: Power the device off.

Returns: void (does not return).

sdk_get_reset_reason(void)

Description: Get the code for the last reset/power-up reason.

Returns: UINT32 - reset-reason code.

sdk_get_reset_reason_string(UINT32 reset_reason_code)

Description: Get a human-readable string for a reset-reason code.

Parameters:

UINT32 reset_reason_code - the code from sdk_get_reset_reason.

Returns: const char* - static description string.

LOG

sdk_log_init_uart(UINT32 port, const SdkUartConfig* config)

Description: Initialise the debug UART used for log output.

Parameters:

UINT32 port - UART port id.
const SdkUartConfig* config - UART configuration.

Returns: SdkResult - 0 success; negative on failure.

sdk_debug_print(const char* format, ...)

Description: Raw debug print (printf-style), no level/tag.

Parameters:

const char* format, ... - printf-style format and arguments.

Returns: void.

sdk_log_info(const char* format, ...)

Description: Emit an INFO-level log line.

Parameters:

const char* format, ... - printf-style format and arguments.

Returns: void.

sdk_log_warning(const char* format, ...)

Description: Emit a WARNING-level log line.

Parameters:

const char* format, ... - printf-style format and arguments.

Returns: void.

sdk_log_error(const char* format, ...)

Description: Emit an ERROR-level log line.

Parameters:

const char* format, ... - printf-style format and arguments.

Returns: void.